

JDBC로 DB 연동과 레이어 분리

오늘의 강의 학습 요약

지난 시간까지 SQL 자체를 학습한 흐름을 바탕으로, 오늘은 ****자바 프로그램에서 DBMS에 접근하는 JDBC(Java Database Connectivity)****를 중심으로 정리되었습니다. JDBC의 핵심 특징(특히 DBMS 독립성과 인터페이스 기반의 일관된 접근 방식)과, 실제 연동을 위해 필요한 **드라이버(JAR) 확보 및 IDE 빌드 패스 설정**이 강조되었습니다.

이어서 JDBC 코딩 절차를 **7단계 흐름**으로 고정해 이해하고, `executeUpdate/executeQuery`의 구분, `ResultSet`의 행/열 처리 방식, 예외 처리(`SQLException`, `ClassNotFoundException`)와 자원 반납(`close`) 원칙을 확립했습니다. 후반에는 오토커밋 해제 기반의 트랜잭션 처리와, 실무형 구조를 위한 **3-Layer(프레젠테이션/서비스/퍼시스턴스)** 분리 및 **DTO 패턴**으로 확장하는 방향까지 다뤘습니다.

1. JDBC 개요와 인터페이스 구조

JDBC는 자바에서 DBMS 종류와 무관하게 동일한 인터페이스와 메소드 패턴으로 데이터베이스에 접속, 질의, 처리할 수 있게 하는 표준 연결 기술입니다.

JDBC는 **Java Database Connectivity**의 약자로, 자바 프로그램이 DBMS와 연결하여 SQL을 실행하고 결과를 받기 위한 표준 기술입니다. 워크벤치/DB버 같은 도구로 SQL을 실행하던 방식에서, 이제는 **자바 코드로 DB에 접근**하게 됩니다.

가장 중요한 성질은 **DBMS 독립성**입니다. MySQL뿐 아니라 Oracle, MSSQL 등 벤더가 달라도 JDBC API를 사용하면 동일한 방식으로 접근할 수 있도록 설계되어 있습니다.

이 "일관된 방법"은 JDBC가 **인터페이스 기반**으로 구성되기 때문에 가능합니다. 예를 들어 `Connection`, `PreparedStatement`, `ResultSet` 등은 `java.sql` 패키지의 인터페이스이며, 각 DBMS 벤더는 이 인터페이스들을 구현한 클래스를 드라이버 형태로 제공합니다.

JDBC에서 자주 맞닥뜨리는 예외는 체크 예외 중심입니다. 특히 DB 연결/SQL 실행 과정에서 `SQLException` 이 발생할 수 있어 반드시 **try-catch** 또는 **throws** 로 처리해야 하며, 드라이버 로딩에서는 `ClassNotFoundException` 처리가 필요합니다.

JDBC는 기본이 **Auto Commit 활성화**입니다. DML을 실행하면 즉시 커밋되어 롤백이 불가능해지므로, 트랜잭션을 묶으려면 오토커밋을 비활성화해야 합니다.

핵심 키워드

#JDBC

#DBMS독립성

#java.sql

#인터페이스

#SQLException

2. 드라이버(JAR) 다운로드와 이클립스 설정

JDBC 연동은 DBMS 벤더가 제공하는 드라이버 JAR을 다운로드한 뒤, 프로젝트 빌드 패스에 등록해야 동작합니다.

JDBC는 표준 API(인터페이스)를 제공하지만, 실제 DBMS별 구현은 각 벤더가 제공합니다. 이 구현체 묶음이 **라이버(Driver) JAR**이며, 이를 프로젝트가 인식해야 JDBC 코드가 실행됩니다.

MySQL의 경우 MySQL 홈페이지에서 **MySQL Connector/J**를 다운로드합니다. 강의에서는 MySQL 8.x 환경에 맞춰 아카이브에서 버전을 맞춘 뒤(예: 8.0.33), ZIP을 내려받아 압축을 풀고 내부의 `mysql-connector-j-...jar` 파일을 확인했습니다. ZIP 자체가 아니라 **JAR 파일**을 빌드 패스에 추가해야 합니다.

이클립스에서 외부 JAR을 연결하는 작업을 **빌드 패스(Build Path)** 설정이라고 합니다. 프로젝트마다 설정해야 하며, 새 프로젝트 생성 시 추가하거나 생성 후 `Configure Build Path` 로 등록할 수 있습니다.

항목	핵심 내용
드라이버 위치	벤더 홈페이지에서 다운로드합니다.
MySQL 드라이버 예	<code>mysql-connector-j-8.0.xx.jar</code> 형태입니다.
등록 대상	ZIP이 아니라 JAR 파일 자체를 등록해야 합니다.
이클립스 등록	프로젝트 우클릭 → Build Path → Configure Build Path → Libraries → Classpath → Add → External JARs 순서입니다.

오라클은 `OJDBC` 등처럼 드라이버 이름이 다르지만, 다운로드 후 빌드 패스 등록이라는 절차는 동일합니다.

핵심 키워드

#드라이버

#Connector/J

#JAR

#Build Path

#Classpath

3. JDBC 7단계 흐름과 예외 처리

JDBC 코드는 7단계 고정 흐름(정보 준비→드라이버 로딩→연결→SQL 준비→전송→결과 처리→자원 반납)으로 구성되며, 대부분 단계에서 예외 처리가 필수입니다.

JDBC는 “이해해서 자유롭게 변형하는 문법”이라기보다, **정해진 절차대로 작성하는 성격**이 강합니다. 핵심은 흐름을 고정 패턴으로 잡는 것입니다.

먼저 접속을 위한 4가지 정보를 준비합니다. 드라이버 클래스명, URL, 사용자 ID, 비밀번호입니다. URL은 `jdbc:mysql://호스트:포트/DB명` 형태이며, 예시로 `localhost:3306/test` 구성이 사용되었습니다.

다음으로 드라이버 로딩은 문자열로 된 클래스명을 메모리에 올리는 작업이며 `Class.forName(driver)` 를 사용합니다. 이때 `ClassNotFoundException` 이 발생할 수 있어 예외 처리가 필요합니다.

DB 연결은 `DriverManager.getConnection(url, user, password)` 로 수행하며 `Connection` 을 얻습니다. 이 시점부터 자바와 DB 사이의 “통로”가 만들어졌다고 보면 됩니다.

SQL은 자바에서 문자열로 작성하되, **JDBC에서는 세미콜론(;)을 붙이면 에러**가 발생할 수 있으므로 제외합니다.

SQL 전달 준비는 `Connection.prepareStatement(sql)` 로 `PreparedStatement` 를 확보합니다. 이후 실제 전송은 SQL 종류에 따라 메소드가 달라집니다.

구분	사용 메소드	반환	의미
DML(INSERT/UPDATE/DELETE)	<code>executeUpdate()</code>	<code>int</code>	변경된 행(Row) 수를 반환합니다.
SELECT	<code>executeQuery()</code>	<code>ResultSet</code>	조회 결과 테이블을 객체로 받습니다.

마지막으로 외부 자원은 반드시 반납해야 하며, 사용 순서의 역순으로 `close()` 해야 합니다. 또한 `close()` 자체도 `SQLException` 을 발생시킬 수 있으므로 예외 처리가 필요합니다.

자원 반납의 대표 역순은 다음과 같이 정리됩니다.

- `ResultSet` 을 사용했다면 먼저 `ResultSet.close()` 를 호출해야 합니다.
- 다음으로 `PreparedStatement.close()` 를 호출해야 합니다.
- 마지막으로 `Connection.close()` 를 호출해야 합니다.
- 자원 반납은 예외 상황에서도 실행되도록 `finally` 에 배치하는 것이 안전합니다.

핵심 키워드

`#Class.forName`

`#DriverManager`

`#Connection`

`#PreparedStatement`

`#executeUpdate`

4. ResultSet 처리: 행 선택과 열 선택

SELECT 결과는 ResultSet으로 받으며, next()로 행을 이동하고 getXxx()로 열 값을 타입에 맞춰 추출합니다.

executeQuery()의 반환값인 ResultSet은 SELECT 결과의 "테이블 형태"를 자바에서 다루기 위한 객체입니다. 처음 ResultSet이 생성되면 현재 위치는 실제 레코드가 아니라 **컬럼 헤더 영역을 가리킨다**고 이해하면 편합니다.

따라서 레코드를 읽으려면 커서를 아래로 내려야 하며, 이 역할을 next()가 수행합니다. next()는 이동 성공 여부를 boolean으로 반환하므로, 레코드 수를 알 수 없는 상황에서 보통 while(rs.next()) 형태로 반복합니다.

열 값 추출은 데이터 타입에 맞는 getXxx()를 사용합니다. 예를 들어 정수는 getInt, 문자열은 getString을 사용합니다. 컬럼 지정은 두 방법이 가능합니다.

지정 방식	예시	주의점
컬럼 헤더명(문자열)	<code>rs.getInt("DPTNO")</code>	별칭(alias)을 쓰면 헤더명이 바뀌므로 결과 헤더를 기준으로 써야 합니다.
컬럼 인덱스(1부터)	<code>rs.getInt(1)</code>	SELECT 결과의 컬럼 순서 기준이며, 테이블 물리 순서가 아닙니다.

특히 별칭을 준 경우, rs.getString("원래컬럼명")이 아니라 **별칭으로 바뀐 헤더명**으로 접근해야 한다는 점이 실수 포인트로 강조되었습니다.

핵심 키워드

#ResultSet

#next

#getInt

#getString

#컬럼별칭

5. DML 처리와 PreparedStatement 파라미터 바인딩

PreparedStatement는 물음표(?) 플레이스홀더를 사용해 동적 값을 바인딩하며, setXxx(위치, 값)으로 값을 치환한 뒤 executeUpdate()로 실행합니다.

INSERT/UPDATE/DELETE는 모두 executeUpdate() 로 실행하며, 반환되는 int 는 영향을 받은 행 수입니다. 이 값으로 성공 여부를 판단할 수 있습니다.

고정 SQL 문자열에 값을 직접 붙이는 방식 대신, ? 플레이스홀더를 두고 실행 직전에 값을 바인딩하는 방식이 권장됩니다. 이는 SQL 문자열 조합 실수를 줄이고, 매개변수 처리 구조를 명확히 합니다.

파라미터 바인딩은 PreparedStatement 의 setXxx() 계열 메소드로 수행하며, 인덱스는 1부터 시작합니다. getXxx() 의 반대 개념으로 이해하면 정리됩니다.

```
String sql = "insert into dpt (dptno, dname, loc) values (?, ?, ?)";
PreparedStatement pstmt = con.prepareStatement(sql);
pstmt.setInt(1, 51);
pstmt.setString(2, "인사");
pstmt.setString(3, "제주");
int n = pstmt.executeUpdate();
```

UPDATE에서도 where 절의 ? 까지 포함하여 위치 인덱스를 정확히 맞춰야 하며, DELETE는 보통 조건부 ? 하나를 바인딩하는 형태로 구성됩니다.

핵심 키워드

#executeUpdate

#플레이스홀더

#setInt

#setString

#DML

6. 트랜잭션 처리: 오토커밋 해제, commit/rollback

JDBC는 기본 오토커밋이므로 트랜잭션을 묶으려면 `setAutoCommit(false)`로 비활성화한 뒤 성공 시 `commit`, 실패 시 `rollback`을 수행해야 합니다.

JDBC는 기본적으로 오토커밋이 켜져 있어 DML이 실행되면 즉시 DB에 반영됩니다. 여러 작업을 하나로 묶고 싶다면 오토커밋을 꺼야 합니다.

트랜잭션 처리의 핵심 흐름은 다음과 같습니다.

- `con.setAutoCommit(false)` 로 자동 커밋을 비활성화합니다.
- 여러 DML을 실행한 뒤 오류가 없으면 `con.commit()` 을 호출합니다.
- 중간에 오류가 발생하면 `catch` 에서 `con.rollback()` 을 호출합니다.
- 커밋/롤백 및 자원 반납 과정도 예외 처리가 필요합니다.

실습에서는 “인sert는 성공, 업데이트는 테이블명 오타로 실패” 같은 의도적 오류를 만들어, insert까지 함께 롤백되어 DB에 반영되지 않음을 확인했습니다.

핵심 키워드

#AutoCommit

#setAutoCommit

#commit

#rollback

#트랜잭션

7. 예외 처리 통합과 finally 자원 반납, NPE 방지

try-catch를 하나로 묶고 **finally**에서 자원을 반납하되, 초기화 실패 시 **close()** 호출로 **NPE**가 나지 않도록 **null** 체크가 필요합니다.

초기 실습에서는 메소드 호출마다 **try-catch** 를 개별로 두어 어떤 단계에서 어떤 예외가 나는지 확인했지만, 이후에는 하나의 **try-catch** 로 묶어 중복을 줄이는 방식으로 정리했습니다.

자원 반납은 반드시 실행되어야 하므로 **finally** 블록에 두는 것이 일반적입니다. 다만 중간에 예외가 발생해 **ResultSet** 이나 **PreparedStatement** 가 생성되지 않았다면 해당 변수는 **null** 일 수 있습니다.

null.close() 를 호출하면 **NullPointerException** 이 발생하며, 이는 런타임 예외로 개발자가 방지해야 하는 오류로 분류됩니다. 따라서 **finally**에서는 **null** 여부를 확인한 뒤 닫아야 합니다.

- **if (rs != null) { rs.close(); }** 형태로 닫아야 합니다.
- **if (pstmt != null) { pstmt.close(); }** 형태로 닫아야 합니다.
- **if (con != null) { con.close(); }** 형태로 닫아야 합니다.
- 예외가 발생하더라도 **close** 로직이 실행되도록 **finally**에 배치해야 합니다.

또한 강의에서는 **main** 에 **throws** 를 붙여 예외를 넘기는 방식은 가능하지만 권장하지 않는다는 점을 언급했습니다.

핵심 키워드

#try-catch

#finally

#close

#NullPointerException

#null체크

8. 3-Layer 아키텍처 분리와 DTO 패턴

JDBC 코드를 프레젠테이션/서비스/퍼시스턴스로 분리하며, 서비스가 커넥션과 트랜잭션을 관리하고 DAO는 DB 작업과 ResultSet 처리 후 DTO 리스트를 반환합니다.

단일 `main` 클래스에 JDBC 코드가 모두 들어가면, 화면 출력, 로직, DB 연동이 한곳에 섞입니다. 이를 실무 형태로 만들기 위해 **3-Layer**로 분리합니다.

레이어별 책임은 명확히 구분됩니다.

레이어	주요 책임	핵심 포인트
Presentation(메인)	입력/출력 등 화면 처리	출력은 DAO가 아니라 메인에서 수행해야 합니다.
Service	비즈니스 로직 + 트랜잭션 처리	커넥션 획득/반납, 커밋/롤백 책임이 핵심입니다.
Persistence(DAO)	DB 연동 및 쿼리 실행	ResultSet 처리 후 DTO로 포장해 반환합니다.

특히 커넥션은 DAO에서 새로 만들지 않고, **서비스에서 1회 생성하여 DAO에 전달**하는 방식이 강조되었습니다. 트랜잭션 처리(커밋/롤백)가 서비스 책임이므로 커넥션을 서비스가 관리하는 구조가 됩니다.

DB에서 가져온 “테이블 결과”를 레이어 간에 전달하려면, 자바에서 표현 가능한 구조로 포장해야 합니다. 이때 사용하는 패턴이 **DTO(Data Transfer Object) 패턴**입니다.

DTO는 테이블의 1행(레코드)을 담는 객체입니다. 예를 들어 `DPTDTO` 에 `dptno` , `dname` , `loc` 필드를 두고, DAO는 `while(rs.next())` 반복마다 DTO를 생성해 값을 넣은 뒤 리스트에 누적합니다.

- DAO는 레코드마다 DTO를 생성해 값을 채우고 리스트에 누적해야 합니다.
- 누적된 리스트는 DAO의 반환값으로 서비스에 전달되어야 합니다.
- 서비스는 DAO 결과를 받아 메인에 그대로 반환하는 흐름을 구성해야 합니다.
- 커넥션은 서비스가 담고, DAO는 자신이 만든 `PreparedStatement` / `ResultSet` 만 달아야 합니다.

서비스 인터페이스/구현체 분리도 함께 적용되었습니다. 예를 들어 `DPTService` (인터페이스)와 `DPTServiceImpl` (구현체)을 두고, 메인은 인터페이스 타입으로 구현체를 받아 다형성을 적용합니다. 또한 DAO를 주입하기 위한 `setDao()` 같은 메소드가 인터페이스에 정의되어야 메인에서 호출 가능하다는 점이 다뤄졌습니다.

핵심 키워드

#3계층

#Service

#DAO

#DTO패턴

#의존성주입